

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

PRACTICAL VI

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 19-08-2026	Date/Time of Submission: 19-08-2026

Aim:-Probability Reasoning (Bayesian Networks & Inference)

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

a)Code:-

```
# Bayesian Network using pgmpy
# Example: Burglary-Alarm Network
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

```
# Step 1: Create Bayesian Network structure
```

```
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
```

```
# Step 2: Define Conditional Probability Tables
```

```
# P(Burglary)
```

```
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
```

```
# P(Earthquake)
```

```
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
```

```
state_names={"Earthquake": ["False", "True"]}
)
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90] # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30], # MaryCalls = False
        [0.01, 0.70] # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
```

```

state_names={
    "MaryCalls": ["False", "True"],
    "Alarm": ["False", "True"]
}
)
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)
# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())
# Step 5: Perform inference
inference = VariableElimination(model)
# Query: Probability of Burglary when John and Mary both call
result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)
print("\nProbability of Burglary given JohnCalls=True and MaryCalls=True:")
print("Ritesh Chaubey T005")
print(result)

```

Output:-

```

Is Bayesian Network valid? True

Probability of Burglary given JohnCalls=True and MaryCalls=True:
Ritesh Chaubey T005
+-----+-----+
| Burglary | phi(Burglary) |
+-----+-----+
| Burglary(False) | 0.7158 |
+-----+-----+
| Burglary(True) | 0.2842 |
+-----+-----+

```

b)Code:-

```

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

```

```
# -----
# Step 1: Define Network Structure
# Relationships: Flu -> Fever, Covid -> Fever, Fever -> Fatigue, Fever -> Cough
# -----
model = DiscreteBayesianNetwork([
    ("Flu", "Fever"),
    ("Covid", "Fever"),
    ("Fever", "Fatigue"),
    ("Fever", "Cough")
])
# -----
# Step 2: Define Conditional Probability Distributions (CPDs)
# -----
# P(Flu): 10% chance a person has the flu
cpd_flu = TabularCPD(
    variable="Flu",
    variable_card=2,
    values=[[0.90], [0.10]], # [P(False), P(True)]
    state_names={"Flu": ["False", "True"]}
)
# P(Covid): 5% chance a person has Covid-19
cpd_covid = TabularCPD(
    variable="Covid",
    variable_card=2,
    values=[[0.95], [0.05]], # [P(False), P(True)]
    state_names={"Covid": ["False", "True"]}
)
# P(Fever | Flu, Covid): Probability of fever based on underlying conditions
# Columns represent combinations: (Flu=F, Covid=F), (Flu=F, Covid=T), (Flu=T, Covid=F),
# (Flu=T, Covid=T)
cpd_fever = TabularCPD(
    variable="Fever",
    variable_card=2,
    values=[
        [0.98, 0.20, 0.15, 0.02], # Fever = False
        [0.02, 0.80, 0.85, 0.98] # Fever = True
    ],
    evidence=["Flu", "Covid"],
    evidence_card=[2, 2],
    state_names={
        "Fever": ["False", "True"],
        "Flu": ["False", "True"],
        "Covid": ["False", "True"]
    }
)
```

```

    }
)
# P(Fatigue | Fever)
cpd_fatigue = TabularCPD(
    variable="Fatigue",
    variable_card=2,
    values=[
        [0.90, 0.15], # Fatigue = False
        [0.10, 0.85]  # Fatigue = True
    ],
    evidence=["Fever"],
    evidence_card=[2],
    state_names={
        "Fatigue": ["False", "True"],
        "Fever": ["False", "True"]
    }
)
# P(Cough | Fever)
cpd_cough = TabularCPD(
    variable="Cough",
    variable_card=2,
    values=[
        [0.85, 0.25], # Cough = False
        [0.15, 0.75]  # Cough = True
    ],
    evidence=["Fever"],
    evidence_card=[2],
    state_names={
        "Cough": ["False", "True"],
        "Fever": ["False", "True"]
    }
)
# -----
# Step 3: Attach CPDs to Network & Validate
# -----
model.add_cpds(cpd_flu, cpd_covid, cpd_fever, cpd_fatigue, cpd_cough)
print("Is Bayesian Network valid?", model.check_model())
# -----
# Step 4: Perform Inference
# -----
inference = VariableElimination(model)
# Query: What is the probability of having Covid if the patient has Fatigue and Cough?
result = inference.query(

```

```

variables=["Covid"],
evidence={
    "Fatigue": "True",
    "Cough": "True"
}
)
print("\nProbability of Covid given Fatigue=True and Cough=True:")
print(result)

```

Output:-

```

Is Bayesian Network valid? True

Probability of Covid given Fatigue=True and Cough=True:
+-----+-----+
| Covid      | phi(Covid) |
+-----+-----+
| Covid(False) | 0.7414 |
+-----+-----+
| Covid(True)  | 0.2586 |
+-----+-----+

```

2. Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

a)Code:-

```

# Posterior Probability using Bayesian Network
# Query: P(Burglary | Alarm = True)
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm")
])
# Step 2: Define probability of Burglary
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
# Step 3: Define probability of Earthquake
cpd_earthquake = TabularCPD(
    variable="Earthquake",

```

```
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
# Step 4: Define probability of Alarm
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# Step 5: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm
)
# Step 6: Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())
# Step 7: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)
print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print("Ritesh Chaubey T005")
print(result)
```

Output:-

```

Is Bayesian Network valid? True

Posterior Probability:
P(Burglary | Alarm = True)
Ritesh Chaubey T005
+-----+-----+
| Burglary      | phi(Burglary) |
+-----+-----+
| Burglary(False) |          0.6264 |
+-----+-----+
| Burglary(True)  |          0.3736 |
+-----+-----+

```

b)Code:-

```

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure (Cause -> Effect)
model = DiscreteBayesianNetwork([
    ("Flu", "Fever"),
    ("Covid", "Fever")
])
# Step 2: Define Prior Probabilities for Causes
cpd_flu = TabularCPD(
    variable="Flu",
    variable_card=2,
    values=[[0.90], [0.10]],
    state_names={"Flu": ["False", "True"]}
)
cpd_covid = TabularCPD(
    variable="Covid",
    variable_card=2,
    values=[[0.95], [0.05]],
    state_names={"Covid": ["False", "True"]}
)
# Step 3: Define Conditional Probability Table for Fever
cpd_fever = TabularCPD(
    variable="Fever",
    variable_card=2,
    values=[
        [0.98, 0.20, 0.15, 0.02], # Fever = False
        [0.02, 0.80, 0.85, 0.98] # Fever = True
    ],
    evidence=["Flu", "Covid"],

```

```
evidence_card=[2, 2],
state_names={
    "Fever": ["False", "True"],
    "Flu": ["False", "True"],
    "Covid": ["False", "True"]
}
)
# Step 4: Add CPDs to model & check validity
model.add_cpds(cpd_flu, cpd_covid, cpd_fever)
print("Is Bayesian Network valid?", model.check_model())
# Step 5: Calculate Posterior Probability P(Covid | Fever = True)
inference = VariableElimination(model)
result = inference.query(
    variables=["Covid"],
    evidence={"Fever": "True"}
)
print("\nPosterior Probability:")
print("P(Covid | Fever = True)")
print("Ritesh Chaubey T005")
print(result)
```

Output:-

```
Is Bayesian Network valid? True

Posterior Probability:
P(Covid | Fever = True)
Ritesh Chaubey T005
+-----+-----+
| Covid      | phi(Covid) |
+=====+=====+
| Covid(False) | 0.7052 |
+-----+-----+
| Covid(True)  | 0.2948 |
+-----+-----+
```